Decision Trees and Random Forest

Chapter 6: **Decision Trees**
Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow, Gurelien Geron, 3e, O'Reilly Media, 2022.

Decision Trees

- Decision trees are versatile machine learning algorithms that can perform both classification and regression tasks.
- They are powerful models capable of fitting complex datasets.
- Decision trees are intuitive, and their decisions are easy to interpret. Such models are often called white-box models.
- One of their key advantages is that they require very little data preparation:
  - No need for feature scaling or normalization
  - Can handle categorical variables (though in practice, libraries like Scikit-learn typically require encoding)

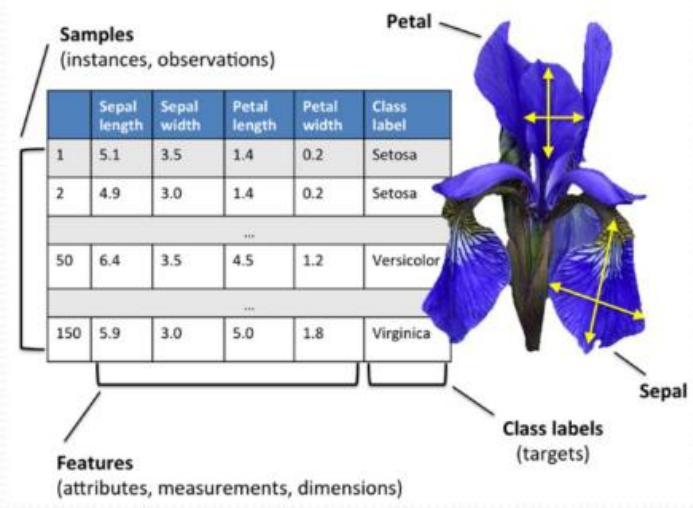
Structure of a Decision Tree

- A decision tree is simply a set of *if–else* rules arranged in a tree-like structure.
- It consists of:
  - Root node: the top-most node representing the first split
  - Split nodes: decision nodes that split the data into subsets
  - Leaf nodes: terminal nodes that provide the final prediction (class label or target value)
- Each split node is defined by a feature and a split value (threshold).
- A feature can be used multiple times in different parts of the tree, possibly with different thresholds.
- The main task in building a decision tree is to decide how to split the data at each internal node.

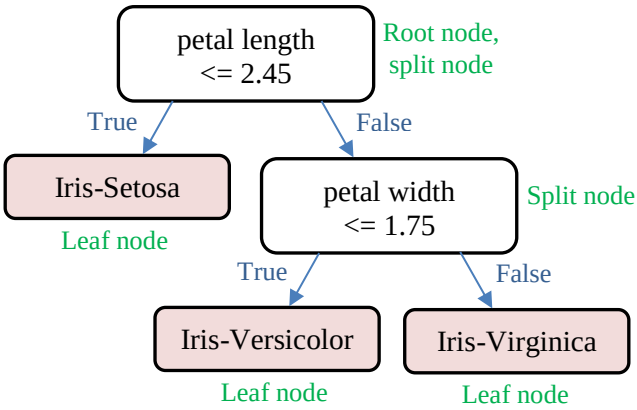
Decision Tree Classifier

- A decision tree can be used for classification tasks, where the target variable is categorical (class labels).
- Instead of predicting a numerical value, the model predicts a class.
- Example

  Consider the Iris dataset, a classic dataset available in Scikit-learn.
  Features: petal length (cm), petal width (cm), sepal length (cm), sepal width (cm)
  Classes: Iris-Setosa, Iris-Versicolor, and Iris-Virginica.
  We use two features: petal length and petal width to build the decision tree.



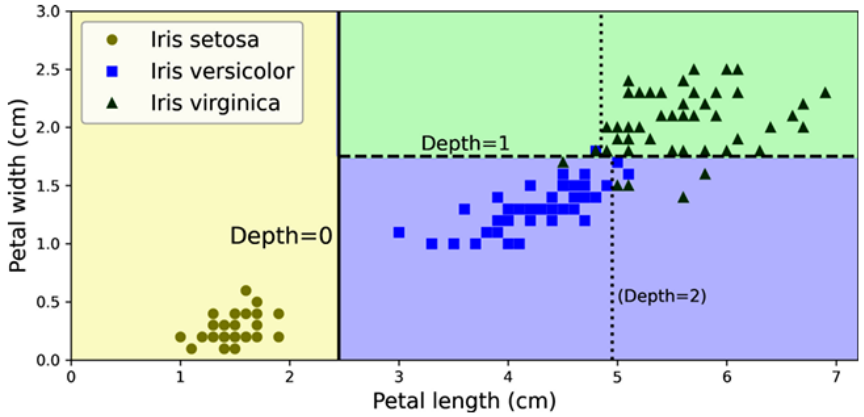
	Sepal length	Sepal width	Petal length	Petal width	Class label
1	5.1	3.5	1.4	0.2	Setosa
2	4.9	3.0	1.4	0.2	Setosa
...					
50	6.4	3.5	4.5	1.2	Versicolor
...					
150	5.9	3.0	5.0	1.8	Virginica



Sample 1: *Input:* petal length = 1.72, petal width = 2.51, *Prediction:* Iris-Setosa
Sample 2: *Input:* petal length = 2.72, petal width = 2.51, *Prediction:* Iris-Virginica

- The tree defines an axis-aligned partition of the feature space.
- Example

  For the Iris dataset example, the following figure shows axis-aligned decision boundaries for the corresponding decision tree. The thick vertical line represents the decision boundary of the root node (depth 0): petal length = 2.45 cm. Since the left hand area is Iris-Setosa only, it cannot be split any further. However, the right hand area at depth-1, splits at petal width = 1.75 cm (represented by the dashed line). The max_depth of the decision tree is 2. If max_depth is set to 3, then the two depth-2 nodes would each add another decision boundary (represented by the two vertical dotted lines)

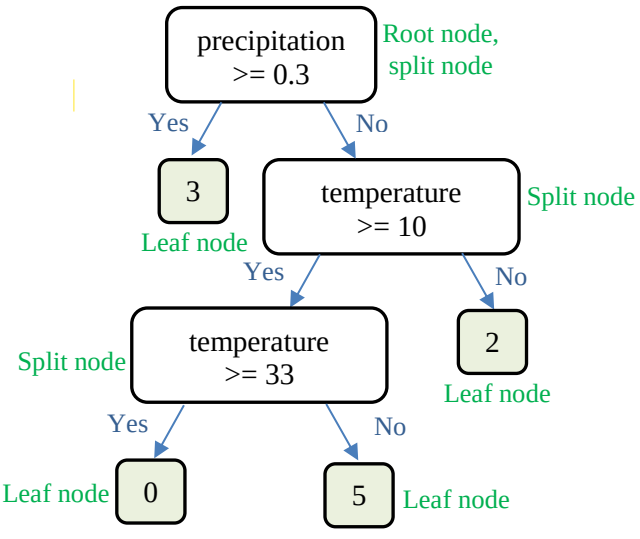


Decision Tree Regressor

- A decision tree can also be used for regression tasks, where the target variable is continuous rather than categorical.
- Instead of predicting a class label, the model predicts a numerical value.
- Example

  We want to predict how many kilometers (km) a person should run based on weather conditions.
  Features: temperature (°C), precipitation (cm/hr)
  Target: distance to run (km)

Draw axis-aligned partition of the feature space for this decision tree.



Learning a Decision Tree

- Two common algorithms used for learning decision trees are CART (Classification and Regression Trees) and ID3 (Iterative Dichotomiser 3).
- The CART algorithm produces binary trees, meaning each decision node splits data into exactly two branches using a threshold.
  Scikit-Learn uses the CART algorithm for decision trees.
- The ID3 algorithm can produce decision trees where nodes may have more than two children, especially when splitting categorical features.

id3 = more than two branches on splitiing

A greedy heuristic means:
At each node, the algorithm chooses the best split right now (locally optimal choice). It does not look ahead to see future splits.

The CART Algorithm

- In its simplest form, the algorithm follows a greedy, top-down process:
  1. Evaluate all possible splits across all features and all candidate thresholds for each feature.
  2. Select the split that results in the best impurity reduction.
  3. Split the dataset into two subsets based on the chosen condition.
  4. Repeat the process recursively for each subset.
  5. Stop when a stopping condition is met.
  6. Assign a class label (classification) or average value (regression) to each leaf node.

An exhaustive search means:
Try all possible trees of all possible structures

- The problem of learning an optimal decision tree (i.e., finding the smallest tree with minimum classification error) is NP-complete, meaning it is computationally infeasible to solve exactly for large datasets.
- Therefore, algorithms like CART use greedy heuristics instead of exhaustive search, making them efficient but not guaranteed to produce the globally optimal tree.

Choosing a Feature

- The algorithm may evaluate all features at each node to find the best split.
- Alternatively, it may evaluate only a subset of features to reduce computation time and help prevent overfitting.

Choosing Candidate Thresholds for a Feature

For Numerical or Continuous Feature

- Sort the feature values, take midpoints between consecutive values. Each midpoint becomes a candidate threshold.
  Example
  Petal length: 1.0, 1.5, 2.0, 2.5, 3.0
  Candidate thresholds: 1.25, 1.75, 2.25, 2.75
- Another way is to look for points where class labels change after sorting. Take midpoint between those two values
  Example
  Values: 1.0 1.5 2.0 2.5 3.0
  Class: A A B B B
  Candidate threshold: average of 1.5 and 2.0 = 1.75

For Binary Features

- Binary features are the easiest.
- There is only one possible split: Feature = 0 / No / False vs Feature = 1 / Yes / True
- So no threshold search is needed.

For Categorical Features

- CART handles categorical features differently depending on implementation.
- One way is to assign one category at a time.
  Example
  Feature: Color = {Blue, Green , Red}
  Candidate threshold: {Red}, {Blue}, {Green}
- Another way as used by Scikit-learn, is to encode categorical features into numbers and then treat like numerical features.

one vs all

Impurity in Decision Trees and Pure Nodes

- Decision trees work by repeatedly splitting data into smaller groups.
- The goal of each split is to make the resulting groups as "pure" as possible.

- A node is called pure if there is no uncertainty in its output, meaning it represents a single class or a single value.
- A pure node is a node in a decision tree where:
  - All training samples belong to the same class (in classification)
  - All target values are identical or nearly identical (in regression)

- Impurity refers to the level of mixing of different classes in a node.
  - A node is highly impure if it contains many different classes in similar proportions.
  - A node is less impure if most samples belong to one class.
  - A node is completely pure if it contains only one class.

there are multiple classes in less impure too

- Decision trees evaluate impurity to decide where to split the data.
- A split is considered good if it reduces impurity.
- In a well-learned decision tree, node impurity generally decreases as we move from the root toward the leaf nodes.
- Common impurity measures include:
  - Gini Impurity (used in CART): Measures how often a randomly chosen sample would be incorrectly classified.
  - Entropy (used in ID3): Measures uncertainty or randomness in the data.
  - Mean Squared Error (for regression): Measures variance of continuous target values.

Stopping Conditions and Pruning in Decision Trees

- Decision trees can easily become very large and complex. This often leads to overfitting.
- To control this, we use two main techniques:
  - Stopping Conditions (Pre-pruning)
  - Pruning (Post-pruning)

Stopping Conditions (Pre-pruning)

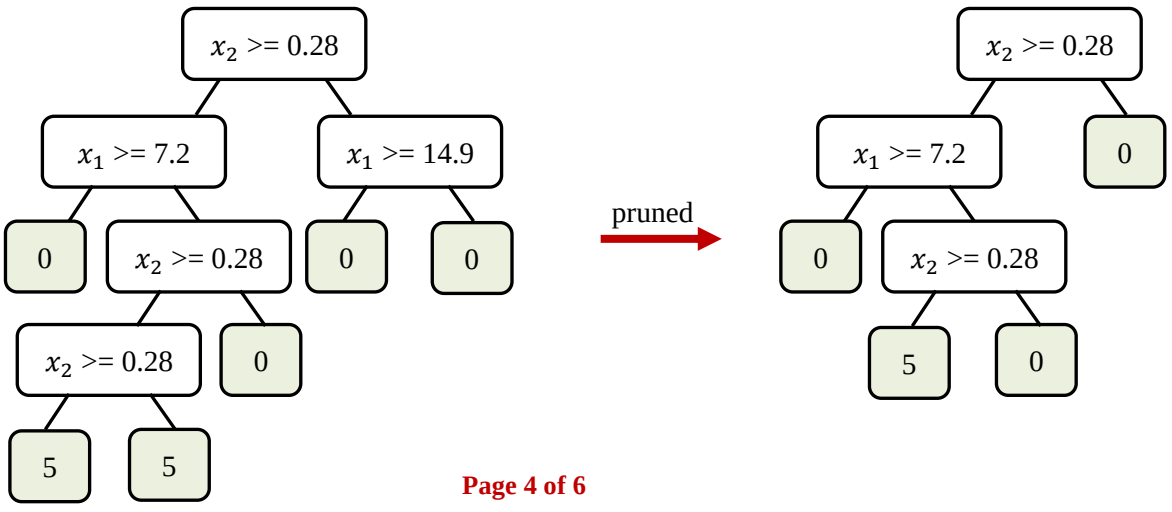
- Stopping conditions decide when to stop growing the tree during training.
- Instead of growing until the tree perfectly fits the data, we stop early to keep the model simple.
- Fast but may stop too early.
- A node stops splitting when:
  - Node is pure: All data points belong to the same class (classification). Or all values are almost the same (regression)
  - No useful improvement: Splitting does not significantly reduce impurity (Gini, entropy, or error)
  - Maximum depth reached: Tree is stopped at a fixed depth to avoid being too large
  - Too few samples: A node has fewer samples than a minimum threshold

Pruning (Post-pruning)

- Pruning is used after the tree has already been built.
- Even if a tree grows fully, it may contain unnecessary branches that reduce performance on new data.
- Pruning removes such branches to make the tree simpler and more general.
- Example

since, both children of x1>=14.9 belongs to class zero, it must be removed

similar for x2>=0.28



Assignment of Target / Class to a Leaf Node

- In a decision tree, once the tree stops splitting, the final nodes are called leaf nodes. Each leaf node represents a final prediction.
- The process of assigning a target value or class label to a leaf node depends on the type of problem:

For Classification Problems

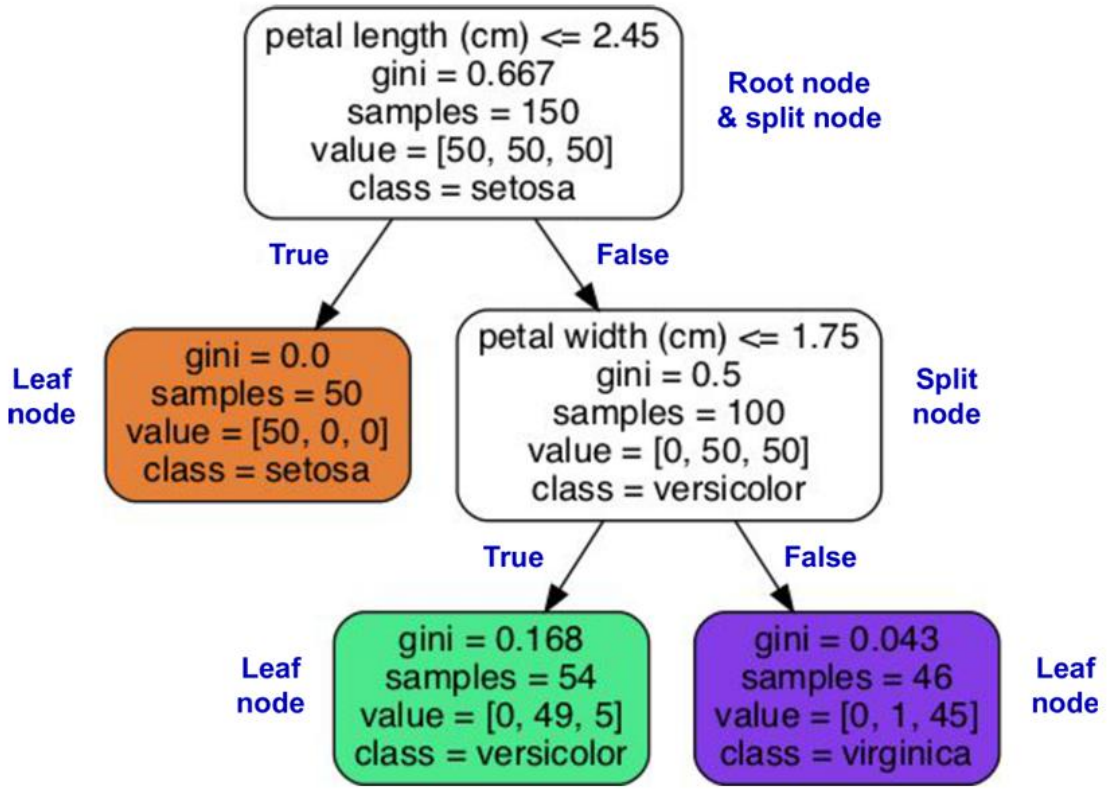
- A leaf node contains a subset of training samples.
- The leaf is assigned the class that appears most frequently among the training samples in that node.
- Example: If a leaf contains 5 samples of Class A, and 3 samples of Class B, then the leaf is labeled as Class A.

For Regression Problems

- A leaf node contains continuous numerical values.
- The leaf value is the average of all target values in that node.
- Example: If a leaf contains values 10, 12, 14, 16, then the predicted value is (10 + 12 + 14 + 16) / 4 = 13

Example

Consider the Iris dataset, a classic dataset available in Scikit-learn.
Features: petal length (cm), petal width (cm), sepal length (cm), sepal width (cm)
Classes: Iris-Setosa, Iris-Versicolor, and Iris-Virginica.
We use two features: petal length and petal width to build the decision tree.
Max-depth of the tree is set to 2.



Impurity Reduction Metrics

- Two common metrics used to measure impurity in classification decision trees are Entropy and Gini Index, while Mean Squared Error (MSE) is used in regression decision trees.
- During tree construction, the algorithm selects splits that reduce impurity; therefore, impurity generally decreases as we move down the decision tree.

Entropy and Information Gain

- Entropy measures the amount of disorder, randomness, or uncertainty in a dataset.
- A node containing samples from many different classes has high entropy.
- A pure node has entropy equal to zero.
- If D is the dataset/subset at any split node containing samples from C classes, entropy is defined as:

$$H(D) = -\sum_{i=1}^{C} P_i \log_2(P_i)$$

Where P_i is the relative frequency (probability) of class i in D.

- Information gain measures the reduction in entropy after a split.
- The split with the highest information gain is preferred.
- If dataset D is split into subsets $D1$ and $D2$, then information gain IF:

$$IG(D,S) = H(D) - \left(\frac{D1}{D}H(D1) + \frac{D2}{D}H(D2)\right)$$

Where S is the split condition, and $D1$ and $D2$ are the number of samples in each subset.

- Entropy after split should be lower than parent entropy, or equivalently, Information Gain should be positive

$$H(D,S) < H(D) \quad \text{or} \quad IG > 0$$

Gini Index

- The Gini index (or Gini impurity) measures how mixed the classes are in a node.
- Lower Gini values indicate purer nodes.
- A pure node has Gini impurity equal to zero.
- If D is the dataset/subset at any split node containing samples from C classes, gini impurity is defined as:

$$gini(D) = 1 - \sum_{i=1}^{C} {P_i}^2$$

Where P_i is the relative frequency (probability) of class i in D.

- If dataset D is split into subsets $D1$ and $D2$, the weighted gini impurity after the split is:

$$gini(D,S) = \frac{D1}{D}gini(D1) + \frac{D2}{D}gini(D2)$$

Where S is the split condition, and $D1$ and $D2$ are the number of samples in each subset.

- The weighted impurity after split should ideally be lower than the parent impurity.

$$gini(D,S) < gini(D) \quad \text{or} \quad \Delta gini = gini(D) - gini(D,S) > 0$$

Mean Squared Error (MSE)

- MSE measures the average squared difference between the actual values and the mean value of a node.
- A node is considered pure in regression if all target values are identical, giving MSE = 0.
- If D is the dataset/subset at any node containing n samples, MSE is defined as:

$$MSE(D) = \frac{1}{n}\sum_{i=1}^{n}\left(y^{(i)} - \bar{y}\right)^2$$

Where: $y^{(i)}$ is the actual target value of sample i, and $\bar{y}$ is the mean of target values in dataset D.

- If dataset D is split into subsets $D1$ and $D2$, then the weighted MSE after the split is:

$$MSE(D,S) = \frac{D1}{D}MSE(D1) + \frac{D2}{D}MSE(D2)$$

Where S is the split condition, and $D1$ and $D2$ are the number of samples in each subset

- The weighted error after split should ideally be lower than the parent error:

$$MSE(D,S) < MSE(D) \quad \text{or} \quad \Delta MSE = MSE(D) - MSE(D,S) > 0$$